Paradigmas de la Programación

Conceptos clave;

1. Paradigma de programación

Un **paradigma de programación** es un modelo o enfoque que define la forma de pensar y estructurar soluciones al implementar un programa. Es una guía conceptual que influye en cómo se diseñan los algoritmos, se estructuran los datos y se organiza el código.

Al comprender los paradigmas imperativo, orientado a objetos, funcional, lógico y otros enfoques declarativos o inductivos, es posible elegir el más adecuado para cada caso práctico, maximizando la eficiencia y claridad del código.

2. Lenguajes de bajo nivel vs. alto nivel

- **Lenguajes de bajo nivel:**
  Se acercan al hardware, lo que permite un control más directo sobre los recursos del sistema (por ejemplo, registros o memoria. Ejemplo: Lenguaje ensamblador.
- **Lenguajes de alto nivel:**
  Ofrecen abstracción, lo que facilita la escritura, comprensión y mantenimiento del código. Ejemplos: Python, Java, C .

3. Clasificación de paradigmas (según Floyd)

Según Robert W. Floyd, los paradigmas de programación se pueden clasificar en tres categorías principales:

Paradigmas que soportan técnicas de bajo nivel (cercanos al hardware).

Paradigmas que utilizan métodos o patrones para el diseño de algoritmos.

Paradigmas que permiten soluciones de alto nivel, enfocándose en la abstracción.

Paradigmas de alto nivel

1. Paradigmas procedimentales u operacionales

Imperativo

El paradigma imperativo se basa en instrucciones explícitas que modifican el estado del programa mediante asignaciones, bucles y estructuras de control.

Características principales:

- Uso de variables, punteros y direcciones de memoria.
- Modificación del estado del programa paso a paso.

Ejemplos: C, Pascal, Modula-2, Fortran.

Orientado a Objetos

Este paradigma organiza el código en clases y objetos, promoviendo la modularidad y reutilización del software.

Características principales:

- Abstracción de datos mediante clases y objetos.
- Gestión automática de memoria (en algunos lenguajes).
- Soporte para herencia (simple o múltiple) y polimorfismo.

Ejemplos: C , Java, C#, VB.NET.

Orientado a agentes

Los objetos (agentes) actúan autónomamente para alcanzar una solución global o local.

Ejemplo: Modelos multiagente como JADE Java Agent Development Framework),

AutoGen...

2. Paradigmas declarativos

El paradigma declarativo se centra en qué debe hacerse, en lugar de especificar cómo hacerlo paso a paso (como ocurre en el paradigma imperativo). Dentro de este grupo encontramos varios subparadigmas:

Funcional

Basado en funciones matemáticas puras.

Características principales:
No tiene efectos secundarios (no modifica variables externas).
Se enfoca en inmutabilidad y composición funcional.

Ejemplos: Haskell, Lisp, Hope.

Lógico

Se basa en lógica matemática mediante predicados.

Características principales:

- Define hechos y reglas para resolver problemas mediante inferencias lógicas.

Ejemplo: Prolog.

3. Otros paradigmas

Paradigmas demostrativos/inductivos

Este enfoque busca generalizar soluciones basándose en observaciones particulares o ejemplos específicos.

Características principales:

Uso de técnicas inductivas para aprender patrones o reglas generales.

Aplicación frecuente en inteligencia artificial y aprendizaje automático.

Ejemplos:

Redes neuronales artificiales.,

Programación genética,

Algoritmos evolutivos.

Basado en formularios

Ejemplo: Hojas de cálculo como Excel.

Paradigma Dataflow

Ejemplo: Apache Nifi, Pentaho Data Integration...

Orientado a eventos

El estado del sistema está determinado por eventos o sucesos que ocurren y cómo estos son manejados.Ejemplo: Hojas de cálculo como Excel, Javascript, c#...